Malware Inspection & Reverse Engineering

Training at Forrest.one · Static and dynamic analysis in an isolated lab



The Forrest.one bear mascot.

How to use this guide

This is a working manual. Every command in it was run against a real file while the guide was written, and every picture is a capture of the tool that produced the output.

Three things to know before you start:

1. You need an isolated lab. Never analyse a sample on a machine you care about. Section 3 builds one.
2. Static before dynamic, always. Read the file before you run it. Most questions are answered without executing anything.
3. Capture what you do. A claim you cannot reproduce is not a finding.

The safety rule that comes before every other rule

Malware analysis has one non-negotiable precondition: the sample must not be able to reach anything you value. Not your files, not your network, not the internet.

Control	Why it exists
Isolated network, no route out	The sample cannot call home or spread
No shared folders from your host	The sample cannot read or encrypt your work
Snapshots before detonation	You can revert to a known-good state
No credentials in the lab	There is nothing worth stealing

Billy Forrest @ https://training.forrest.one

© Billy Forrest @ https://training.forrest.one

Control	Why it exists
Samples are never executed on the host	The host is the one machine that must stay clean

Everything in this guide runs inside containers on an internal-only Docker network. The host never executes a sample.

A note on what you analyse. The samples in this guide are inert and synthetic. They are real, structurally valid PE and ELF files, but they carry indicators rather than payloads. Where a demonstration needs real behaviour, the guide uses a purpose-built benign specimen whose source is included — see samples/beacon_sim.c. Never execute a file you did not build or obtain from a trusted source.

1. Why malware is worth reading

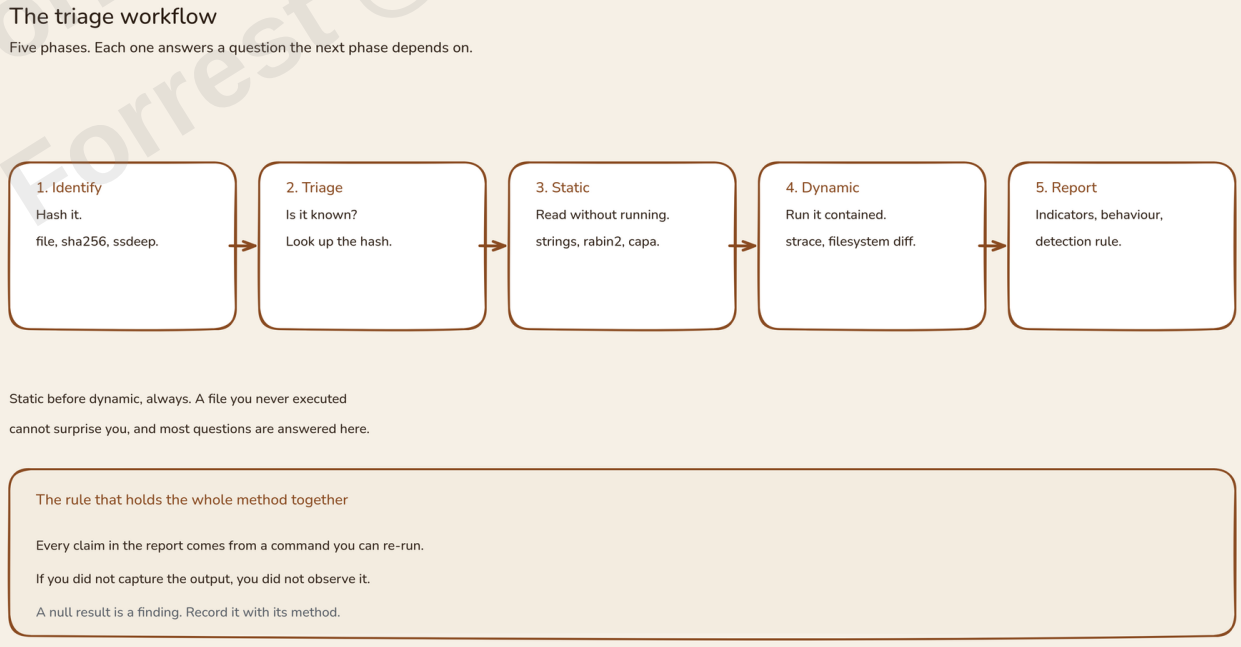
Malware is somebody else's software, written to solve a problem you would not want to have. Reverse engineering it answers four questions:

Question	What the answer gives you
What can it do?	Capabilities, ranked by the risk they represent
How does it persist?	The keys, services and files to remove
Where does it call home?	Network indicators you can block
How do I recognise it again?	A detection rule that outlives the hash

Hashes are perishable, behaviour is durable. Changing one byte changes the hash, so a hash-only detection fails against the next build. A rule built on behaviour — a mutex name, a pipe name, a sequence of API calls — survives repackaging. That is why this guide spends most of its length on behaviour and only a paragraph on hashing.

2. The workflow

Five phases, in order. Each answers a question the next phase depends on.



Billy Forrest @ https://training.forrest.one

© Billy Forrest @ https://training.forrest.one

Figure 1: The five phase triage workflow, from identification to report.

Phase	Question	Output
1. Identify	What is this file?	Hash, type, size
2. Triage	Is it already known?	A verdict or a reason to continue
3. Static	What does it do without running?	Strings, imports, capabilities
4. Dynamic	What does it do when it runs?	Syscalls, files, network
5. Report	What does the reader act on?	Indicators and a detection rule

Why static first. A file you never executed cannot surprise you. Static analysis is also fast: most questions a triage analyst has are answered in the first two phases.

3. Build the lab

The lab is two containers on a Docker network with no gateway and no DNS. The analyst container holds the tooling; the victim container is where a sample is detonated.

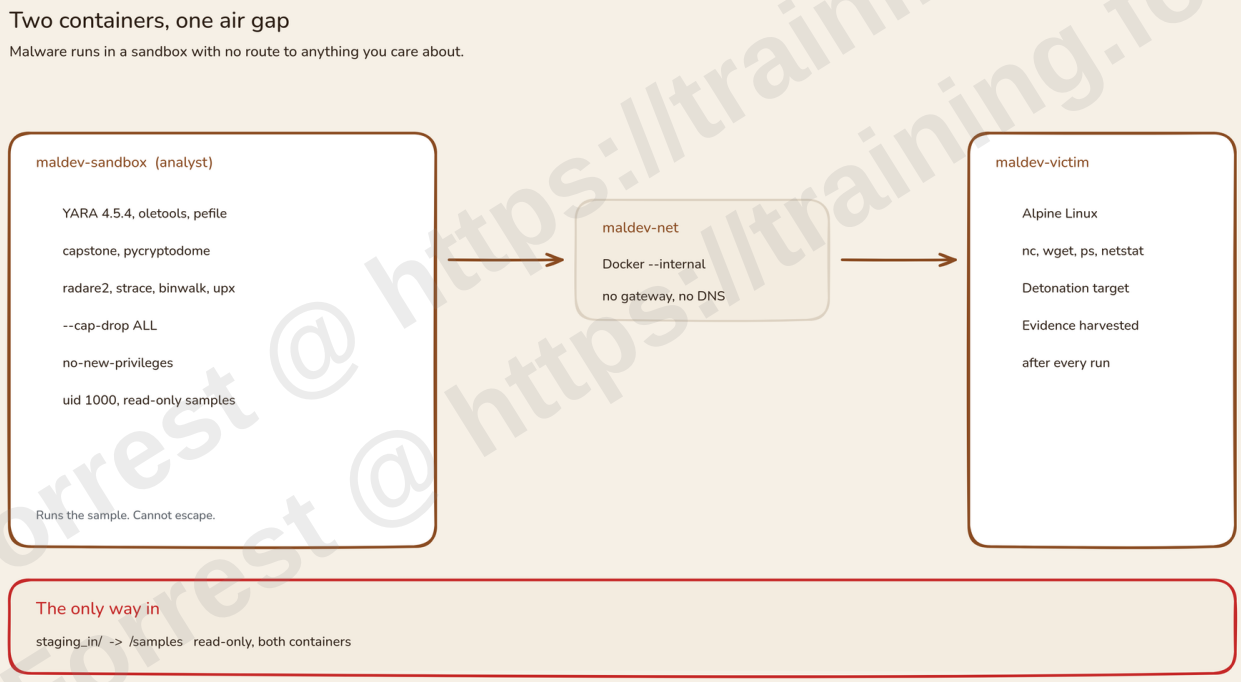


Figure 2: The analyst container, the internal network, and the victim container.

Starting and verifying the lab

```
cd /home/billy-forrest/Malware/sandbox
./sandbox.sh up        # start, then verify isolation
./sandbox.sh verify    # isolation assertions — run before EVERY dynamic analysis
./sandbox.sh status    # what is running
./sandbox.sh logs      # harvest victim evidence to evidence/<timestamp>/
./sandbox.sh down      # stop
```

verify must print all six PASS lines. If it does not, stop and fix it:

Billy Forrest @ https://training.forrest.one

© Billy Forrest @ https://training.forrest.one

```
PASS  analyst: no internet
  PASS  analyst: no LAN
  PASS  analyst: no DNS
  PASS  analyst: toolchain
  PASS  analyst: strace runs
  PASS  victim: no internet
ALL CHECKS PASSED — sandbox is safe to detonate in
```

The air gap

Samples enter through exactly one directory, mounted read-only into both containers:

```
/home/billy-forrest/Malware/sandbox/staging_in/   ->   /samples   (read-only)
```

Nothing else of the host is visible to a container that runs malware.

The hardening, and why it is not negotiable

The containers run with --cap-drop ALL, --security-opt no-new-privileges, a non-root uid, pids and memory limits, and a nosuid,nodev,noexec /tmp.

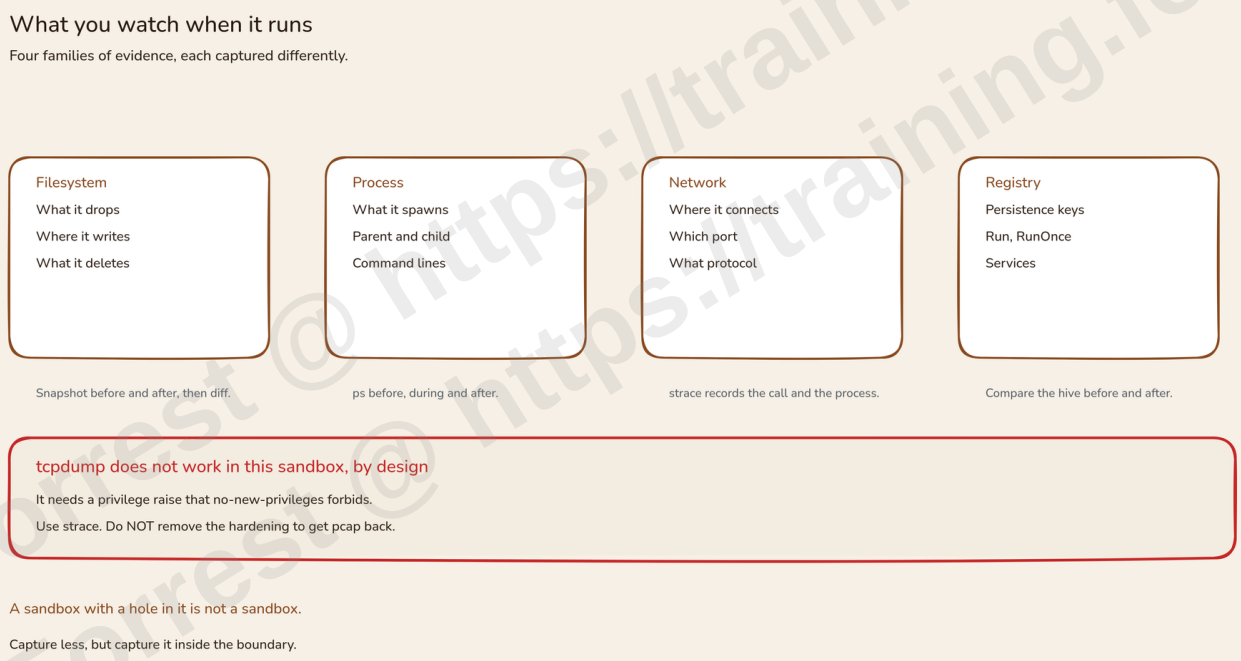


Figure 3: What you watch when a sample runs, and the tcpdump caveat.

tcpdump does not work in this sandbox, by design. It ships file capabilities and needs a privilege raise to apply them, which no-new-privileges forbids. CapEff stays zero even with --cap-add NET_RAW. Use strace instead — it records the process name alongside the connection, which is arguably better for attribution. Do not remove no-new-privileges to get pcap back. That is a real isolation loss traded for a convenience you can get another way.

Working in the analyst container

```
./sandbox.sh analyst     # shell as the analyst
./sandbox.sh victim      # shell as the victim
```


© Billy Forrest @ https://training.forrest.one

The analyst toolchain lives in a venv on a bind-mounted volume, so it survives a container rebuild:

```
/opt/maldev-venv/bin/python     # pefile, capstone, yara, oletools, pycryptodome
/opt/maldev-tools/strace        # strace is NOT on PATH; call it by full path
```

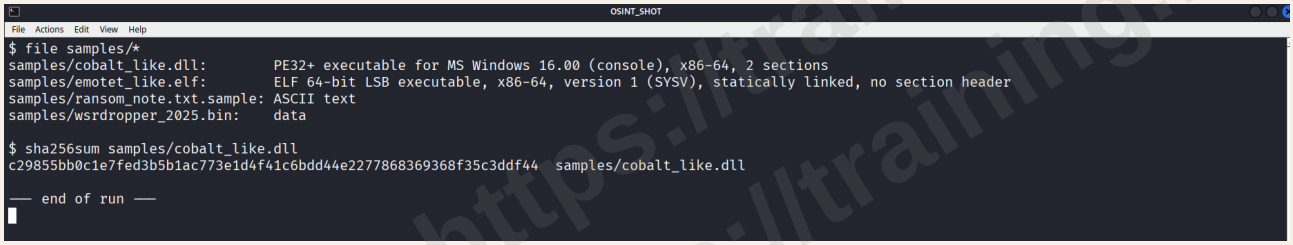
strace is not on PATH. command -v strace returns nothing and a bare strace fails with "command not found". Call /opt/maldev-tools/strace explicitly. This trips up every first-time user of the sandbox.

4. Phase 1 — Identify

4.1 Hash it first

Everything downstream is keyed to the hash. Compute three, because different services index different ones.

```
sha256sum samples/cobalt_like.dll
md5sum    samples/cobalt_like.dll
sha1sum   samples/cobalt_like.dll
ssdeep    samples/cobalt_like.dll   # fuzzy hash: survives small edits
```



Screenshot: file type identification and SHA-256 for each sample in the set.

What the run above shows. The set contains four files of genuinely different types:

File	Identified as	What it tells you
cobalt_like.dll	PE32+ executable, x86-64	Windows, 64-bit, two sections
emotet_like.elf	ELF 64-bit, statically linked, no section header	Linux, and the missing section header is itself a signal
wsrdropper_2025.bin	data	No recognised format — unpack it or carve it
ransom_note.txt.sample	ASCII text	A document, not an executable

"No section header" is worth pausing on. A statically linked ELF whose section headers have been stripped has been through a tool. Section headers are not required to run, so removing them costs the malware nothing and costs you the symbol and section names you would otherwise read.

ssdeep earns its place. A fuzzy hash produces a similarity score between two files. If a sample is a lightly modified variant of something you have seen, ssdeep tells you that when SHA-256 says the files are unrelated.

4.2 Is it already known?

Look the hash up before you spend an hour reversing it. Public services index millions of samples.

© Billy Forrest @ https://training.forrest.one

Service	What it answers
VirusTotal	Multi-engine verdict, community comments
MalwareBazaar	Whether the sample is already collected and tagged
Hybrid Analysis	Existing sandbox reports
Any.Run	Interactive existing analysis

If the hash is unknown, that is information, not a dead end. It means either a genuinely new sample or a trivially recompiled old one. Both are worth continuing.

Never upload a confidential sample to a public service. A report submitted to a multi-scanner can become visible to other users. Check your handling rules first.

5. Phase 2 — Static analysis

Static analysis reads the file without executing it. This is where most of your answers live.

5.1 strings — the cheapest win in the discipline

```
strings -n 6 samples/cobalt_like.dll | head -22
```



Screenshot: strings output from the cobalt-like sample showing a C2 URL, a named pipe, a user agent and a jitter setting.

What the run above shows is a complete picture of the implant's configuration, extracted in one command:

String	What it means
https://cdn.static-resources.test/microsoft-ph-update	The C2 endpoint, disguised as an update path
\.\pipe\MSSE-2679-server	A named pipe — Cobalt Strike's default peer-to-peer channel
Mozilla/5.0 (Windows NT 10.0; Win64; x64)	A user agent, for blending into normal traffic
jitter 25%	Beacon jitter, the classic sleep-timing evasion
XfwdXHpbpcGVlTVNTRbyNjc5LXhIcnZlcg==	A base64 blob — decode it

That is the whole detection story from one command. A URL, a pipe name, a user agent, a beacon setting and an encoded blob. Before you disassemble a single instruction, you know what this file is for.

Decode the base64 immediately. Encoded strings are where configuration hides:

```
echo 'XfwdXHpbpcGVlTVNTRbyNjc5LXhIcnZlcg==' | base64 -d
```

Read strings output in this order: URLs and IPs, then paths, then registry keys, then user agents, then anything that looks encoded. URLs and IPs are your network indicators and they are the most actionable.

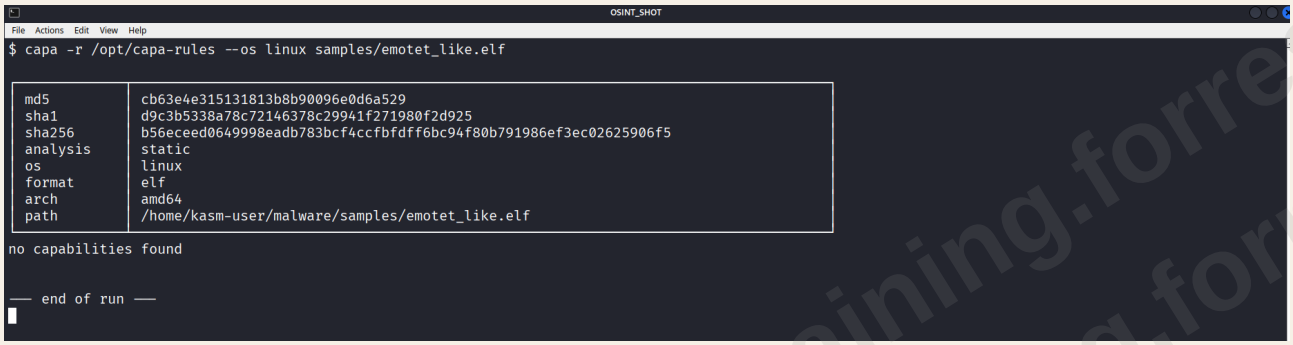
© Billy Forrest @ https://training.forrest.one

When strings shows nothing useful, the strings are obfuscated. See the anti-analysis section below — this is a finding, not a failure.

5.2 rabin2 — parse the executable structure

strings reads raw bytes. rabin2 understands the file format, so it tells you what the loader will do.

```
rabin2 -I samples/cobalt_like.dll     # headers
rabin2 -i samples/cobalt_like.dll     # imports
rabin2 -S samples/cobalt_like.dll     # sections
rabin2 -zz samples/cobalt_like.dll    # strings, with offsets
```



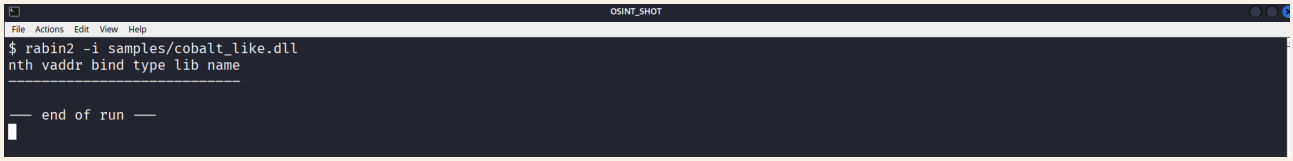
Screenshot: rabin2 parsing the PE header fields of the sample.

What the run above shows. The header-level facts that classify the file:

Field	Value	Reading
arch	x86	32-bit, so it blends into a mixed environment
bits	32	Confirms the above
bintype	pe	Windows portable executable
canary	false	Compiled without stack protection — normal for malware
os	windows	The target platform
stripped	true	Symbol names removed
subsys	Windows CUI	Console subsystem
lang	c	Written in C
compiler	unset	The build toolchain is not identifiable

compiler being unset is a useful negative. Recognisable compilers leave fingerprints — a Delphi or Go binary is often identifiable at a glance. An unidentified toolchain often means a custom or stripped build, which tends to correlate with purpose-built malware rather than commodity crimeware.

```
rabin2 -i samples/cobalt_like.dll
```



Screenshot: the import table, showing which Windows API functions the sample calls.

© Billy Forrest @ https://training.forrest.one

The import table is the single strongest static signal you have. An import is a statement of intent: a program cannot call a function it did not import.

Import	Capability it implies
CreateProcess	Spawns another program
VirtualAlloc	Allocates executable memory — often for a payload
WriteProcessMemory	Writes into another process — injection
CreateRemoteThread	Runs code in another process — injection
InternetOpen / WinHttp	Network communication
RegSetValue	Writes the registry — **likely** persistence
CryptEncrypt	Cryptographic operations — **likely** ransomware or C2 crypto

VirtualAlloc plus WriteProcessMemory plus CreateRemoteThread together is the classic injection triad. Any binary importing all three deserves a much closer look.

On the sample above, the import list is short. Remember that: a short import table on a binary that clearly does something is a signature of packing, which is the next thing to check.

5.3 Sections and entropy — finding what is hidden

Packed and encrypted data has **high** entropy. Measuring it tells you whether there is something underneath the visible file.

```
xxd samples/wsrdropper_2025.bin | head -14
```



Screenshot: hex dump of the dropper sample showing its structure and encoded region.

What the run above shows. A header region in readable form, then a block of bytes that is not readable text. On a real sample this pattern — a plausible header followed by an opaque blob — means the interesting part is in the blob.

Read a hex dump like this:

What you see	What it means
A filename followed by readable text	An embedded or appended file
Long runs of random-looking bytes	Compressed or encrypted data
A repeating structure	An encoded buffer or a resource table
MZ (4d 5a) in the middle of a file	**An embedded executable** — carve it

Finding MZ inside a file is the moment to stop and carve. A file with another executable inside it is a dropper by definition.

© Billy Forrest @ https://training.forrest.one

```
# find every embedded PE in a file
binwalk samples/wsrdropper_2025.bin

# extract what binwalk finds
binwalk -e samples/wsrdropper_2025.bin

# carve a specific region once you know the offset
dd if=samples/wsrdropper_2025.bin of=carved.bin bs=1 skip=<offset>
```

Entropy in one command. Anything above roughly 7.0 is compressed or encrypted:

```
binwalk -E samples/wsrdropper_2025.bin
```

A section with **high** entropy and no corresponding imports is packed. Unpack it before you spend time disassembling, because a disassembler reading packed bytes produces confident nonsense.

5.4 The PE structure you are reading

Every field above comes from the same fixed layout. Learn it once and rabin2 output becomes readable at a glance.

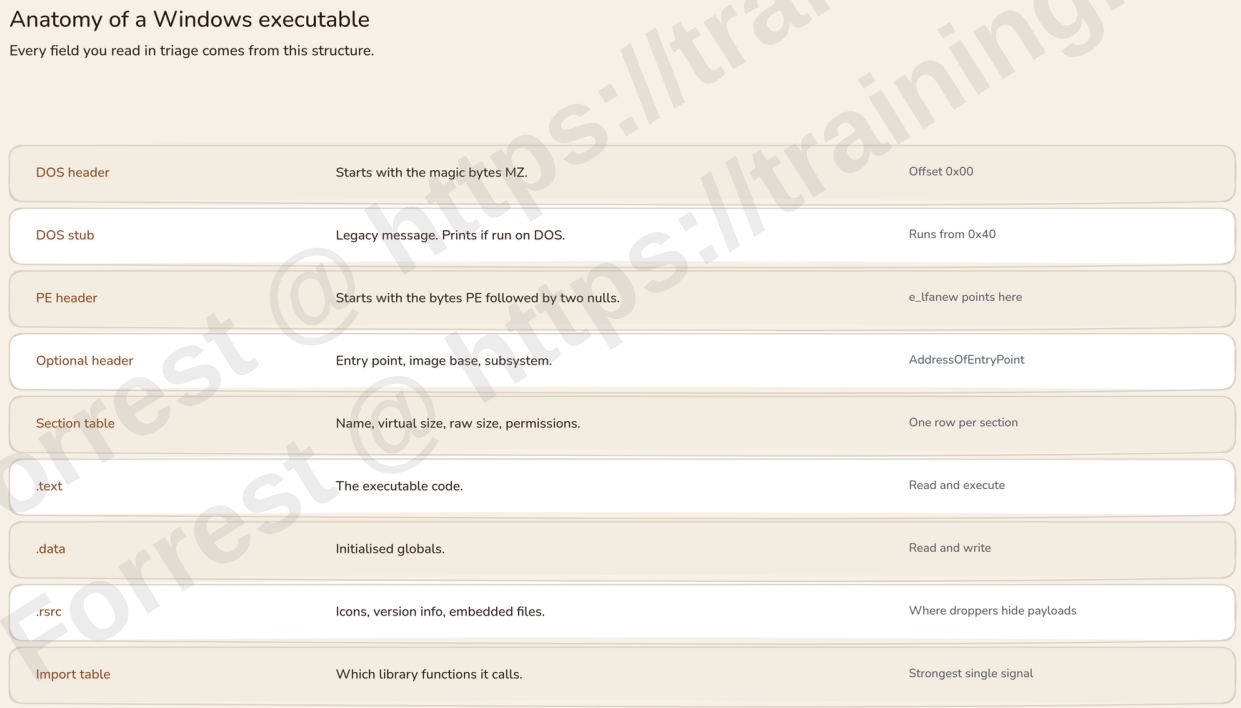


Figure 4: The structure of a Windows executable, field by field.

Field	What it tells you
DOS header (MZ)	The file is a DOS-era format with a modern extension
e_lfanew	The offset where the real PE header starts
PE header (PE\0\0)	The signature that confirms a PE file
Optional header	Entry point, image base, subsystem
Section table	Name, size and permissions of each section
.text	Executable code

© Billy Forrest @ https://training.forrest.one

Field	What it tells you
.data	Initialised globals
.rsrc	Icons, version info, **embedded files**
Import table	The library functions it calls

Two things worth internalising. First, the PE\0\0 signature is four bytes — the two characters followed by two nulls — and a tool that "fixes" it is trying to stop a scanner recognising the file. Second, .rsrc is where droppers hide payloads, because it is designed to hold arbitrary binary data and rarely attracts attention.

5.5 capa — what the program can do

Writing capability rules by hand does not scale. capa matches the file against a rule set and reports capabilities in plain language, mapped to MITRE ATT&CK.;

```
capa -r /opt/capa-rules --os linux samples/emotet_like.elf
```



Screenshot: capa identifying the file and reporting its analysis metadata.

What the run above shows, and why the null result matters. capa identified the file correctly — format elf, arch amd64, analysis static — and then reported no capabilities found.

That is a correct answer, not a failure. The synthetic sample is a header scaffold: it carries indicators for strings and YARA to find, but it has no compiled code for capa to reason about. Reporting it as a clean negative, with the method stated, is exactly what an analyst should do.

Two capa facts that cost time if you do not know them:

- --os is often required. Without it, capa fails with "Input file does not appear to target a supported OS" on files it can otherwise analyse perfectly.
- The PyPI install ships no rules. capa installed via pipx reports "default embedded rules not found" and needs -r /opt/capa-rules. The standalone release binary embeds them; the Python package does not.

© Billy Forrest @ https://training.forrest.one

On a real binary, the same command produces the intelligence you want:

Capability	Namespace
query environment variable	host-interaction/environment-…
enumerate files on Linux	host-interaction/file-system
write file on Linux	host-interaction/file-system
get hostname	host-interaction/os/hostname
terminate process	host-interaction/process
get password database entry on Linux	host-interaction/session

ATT&CK Tactic	ATT&CK Technique
DISCOVERY	File and Directory Discovery [E1083]
DISCOVERY	System Information Discovery [E1082]
FILE SYSTEM	Get File Attributes [C0049]
FILE SYSTEM	Writes File [C0052]
PROCESS	Terminate Process [C0018]

Read the ATT&CK; table, not just the capability list. The capabilities tell you what the code does; the ATT&CK; mapping tells you what that means. "Get password database entry" under host-interaction/session is credential access, and it escalates the finding from "interesting" to "report it today".

5.6 floss — strings that strings cannot see

Obfuscated strings are decoded at run time, so a static strings sweep misses them. FLOSS emulates the decoding routines and recovers them.

```
floss samples/cobalt_like.dll
```



Screenshot: FLOSS extracting static and decoded strings from the sample.

What the run above shows. FLOSS reports its phases — static strings, stack strings, tight strings, then string decoding — and then summarises: 7 static strings, and 0 decoded strings. For this sample the decoded count is zero because the strings are stored plainly.

FLOSS is PE and shellcode only. On an ELF it fails immediately:

```
ERROR: floss: failed to analyze sample: FLOSS currently supports the following
formats for string decoding and stackstrings: PE
```


© Billy Forrest @ https://training.forrest.one

That is a hard limitation of the tool, not a mistake in your command. For ELF binaries, use strings, rabin2 -zz, and manual decoding.

The three string classes FLOSS distinguishes are worth knowing:

Class	What it is
Static	Plain strings in the binary — what strings finds
Stack	Built on the stack one character at a time, to defeat strings
Decoded	Encrypted, decrypted by a routine FLOSS emulates

A binary with many stack strings and few static ones is deliberately hiding its text.

5.7 YARA — writing the detection

Once you have indicators, a YARA rule turns them into something reusable. YARA matches patterns across a corpus, so one rule can screen thousands of files.

```
yara -w yara_rules.yar samples/
```



Screenshot: YARA matching five rules across the synthetic sample set.

What the run above shows. Five matches across four files:

```
Simulated_Emotet_Downloader          samples/emotet_like.elf
Simulated_CobaltStrike_Beacon        samples/cobalt_like.dll
Simulated_Benign_PE_Baseline         samples/cobalt_like.dll
Simulated_Ransomware_Decryptor_Note  samples/ransom_note.txt.sample
Simulated_Dropper_Stage              samples/wsrdropper_2025.bin
```

Note that one file matched two rules. cobalt_like.dll matched both the beacon rule and the benign baseline rule. That is not a bug — it is the rule set working as designed, and it teaches the lesson every detection engineer learns: overlapping rules create ambiguity, and ambiguity in a detection is a false positive waiting to happen. Resolve it by making the specific rule more specific, not by deleting the general one.

Billy Forrest @ https://training.forrest.one

© Billy Forrest @ https://training.forrest.one

A rule you can read

```
rule Simulated_Emotet_Downloader {
    meta:
        family      = "Emotet-style downloader"
        category    = "simulated"
        description = "C2 beacon + persistence under HKCU Run"
        harmless    = true

    strings:
        $c2       = "update.cdn-sync.test"          ascii wide
        $mutex    = "MSUpdate_8f3a91"               ascii wide
        $persist  = "CurrentVersion\\Run"           ascii wide
        $beacon   = "C2 beacon every 60s"           ascii wide

    condition:
        uint32(0) == 0x464c457f and                 // ELF magic
        ( 2 of ($c2, $mutex, $persist, $beacon) )
}
```

Read the condition clause carefully — it is where the engineering is:

- uint32(0) == 0x464c457f anchors the rule to ELF files by magic bytes, so it cannot fire on Windows binaries that happen to contain one of the strings.
- 2 of (...) requires two indicators, not one. A single-string rule fires on every file that contains that string for innocent reasons. Requiring two is the difference between a rule you can deploy and a rule that pages you at 3am for a false positive.

Write the meta: block properly. author, description, date and a reference make a rule maintainable a year later, when nobody remembers why it exists.

YARA-X is the successor to YARA. VirusTotal rewrote YARA in Rust; it is production- ready, roughly 99% rule-compatible, and substantially faster. Rules written for YARA mostly run unchanged, so anything you write today is not wasted.

5.8 Anti-analysis — when the file fights back

The techniques below are why static analysis sometimes stalls. Recognising them is the first half of defeating them.

Billy Forrest @ https://training.forrest.one

© Billy Forrest @ https://training.forrest.one

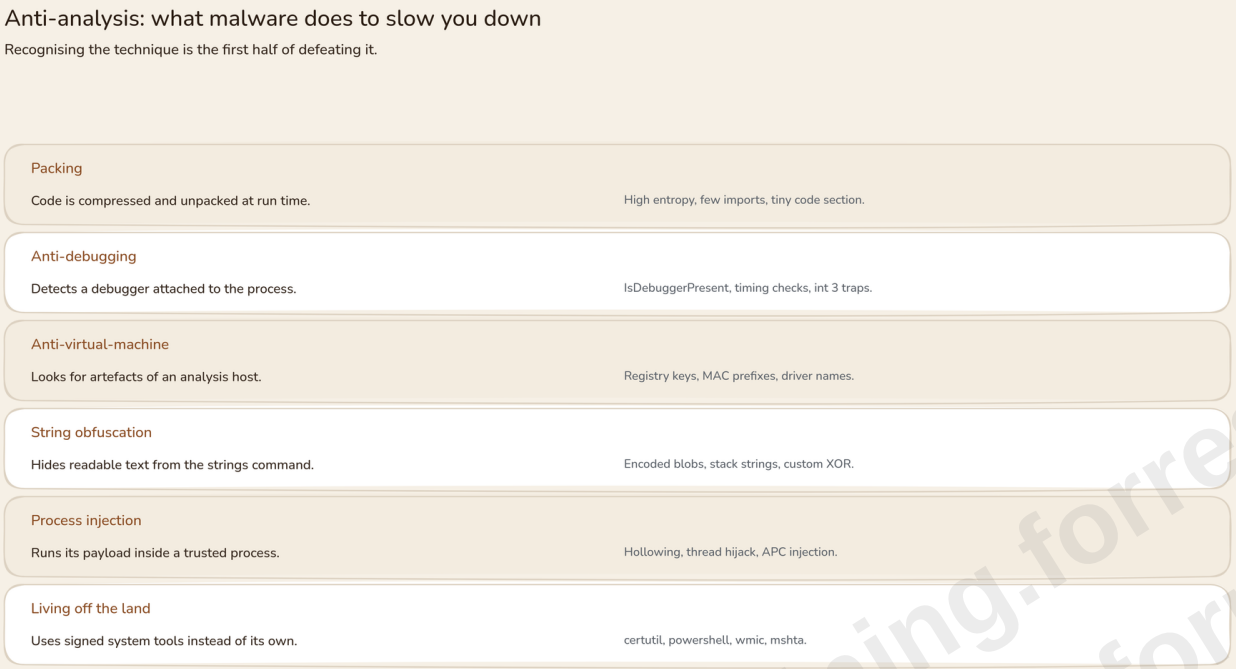


Figure 5: Common anti-analysis techniques and the tell for each one.

Technique	What it does	The tell
Packing	Compresses code, unpacks at run time	**High** entropy, few imports, tiny .text
Anti-debugging	Detects an attached debugger	IsDebuggerPresent, timing checks, int 3
Anti-VM	Detects an analysis host	Registry keys, MAC prefixes, driver names
String obfuscation	Hides text from strings	Encoded blobs, many stack strings
Process injection	Runs payload in a trusted process	The injection triad in the imports
Living off the land	Uses signed system tools	certutil, powershell, wmic, mshta

Packing is the most common, and it is defeatable. Options, in order of effort:

1. Unpack with upx if it is UPX. upx -d packed.exe. UPX is a legitimate compressor, so its output is trivially reversible. A UPX-packed malware sample is a **low**-effort build.

   bash upx -d samples/suspected_packed.exe

   Let it unpack itself. Run it in the sandbox, let it decompress into memory, then dump the process memory and analyse that. This is the reliable general answer.

   Find the original entry point. The unpacker stub must jump to the real code eventually. Break there and dump.

   Living off the land is the hardest to detect and the easiest to write. A script that calls certutil to download a file produces no malicious binary at all. Detection has to move from file signatures to command-line monitoring — which is a different kind of control from the ones this guide otherwise teaches.

© Billy Forrest @ https://training.forrest.one

6. Phase 3 — Dynamic analysis

Dynamic analysis runs the sample and watches what it does. It answers the questions static analysis cannot: what it actually touches, and in what order.

Never skip the isolation check. ./sandbox.sh verify first, every time.

6.1 The benign specimen

The synthetic samples are indicator carriers with no compiled code, so they have no runtime behaviour to observe. To demonstrate dynamic analysis honestly, the guide includes a small benign specimen you build yourself:

```
samples/beacon_sim.c      ->  beacon_sim      (789,224 bytes, statically linked)
sha256 029f9b26561d00f276a93a9dcf1699a3fba8ca00578079e21d813cecf3d4d153
```

It performs four things an analyst looks for, and nothing else:

Behaviour	Why it is in there
Writes /tmp/.beacon_sim_drop	Dropper behaviour
Reads /etc/passwd	Credential access
Connects to 192.0.2.10:8443	C2 beacon attempt
Forks and execs /bin/echo	Process spawning

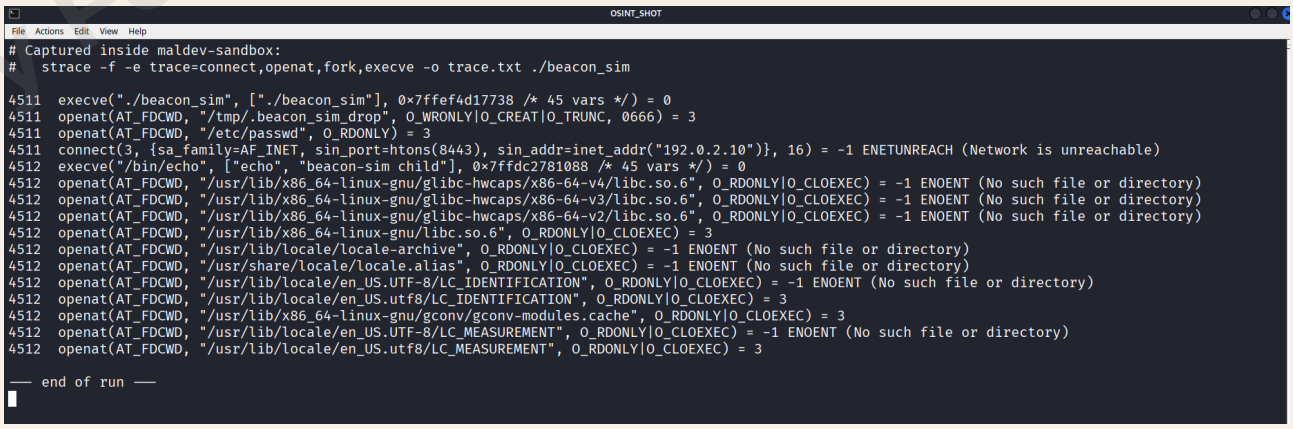
192.0.2.10 is in TEST-NET-1, reserved by RFC 5737 for documentation. It is guaranteed non-routable, so the specimen cannot contact anything even if the sandbox were misconfigured — a second layer of safety independent of the network rules.

There is no payload, no persistence and no evasion. Its whole purpose is to give the instrumentation something real to observe.

6.2 Trace it

strace records every system call a process makes, with the calling process id. In this sandbox it is the primary network-capture tool, because tcpdump cannot run.

```
cd /samples
/opt/maldev-tools/strace -f -e trace=connect,openat,fork,vfork,execve \
  -o /tmp/trace.txt ./beacon_sim
```



Screenshot: strace output captured inside maldev-sandbox, showing the file writes, the credential read, the blocked connection and the child process.

© Billy Forrest @ https://training.forrest.one

Read the run above line by line. This is the whole of dynamic analysis in sixteen lines:

Line	Reading
execve("./beacon_sim", ...)	The process starts — your baseline reference
openat(..., "/tmp/.beacon_sim_drop", O_WRONLY|O_CREAT|O_TRUNC, 0666) = 3	**Drops a file.** Write access, created if absent, truncated if present
openat(..., "/etc/passwd", O_RDONLY) = 3	**Reads the account database.** Credential access
connect(3, {sin_port=htons(8443), sin_addr=inet_addr("192.0.2.10")}, 16) = -1 ENETUNREACH	**Beacon attempt, blocked by the network**
execve("/bin/echo", ["echo", "beacon-sim child"], ...)	**A child process.** The 4512 on this line is a different pid from 4511 above

The ENETUNREACH on the connect line is the single most important line in this guide. It proves two things at once: the sample tried to reach the network, and the network was not there to reach. That is the isolation working, evidenced from inside the analysis rather than asserted from outside it.

The pid column is not decoration. 4511 writes the files and makes the connection; 4512 is the child. Following the pid is how you attribute an action to the right process when a sample spawns children — which is exactly what malware does.

Reading an openat line:

```
openat(AT_FDCWD, "/etc/passwd", O_RDONLY) = 3
       │          │             │          └─ return: fd 3, or a negative errno
       │          │             └─ flags: O_RDONLY, O_WRONLY, O_CREAT...
       │          └─ the path
       └─ directory reference, AT_FDCWD means the current directory
```

6.3 What to watch, and how

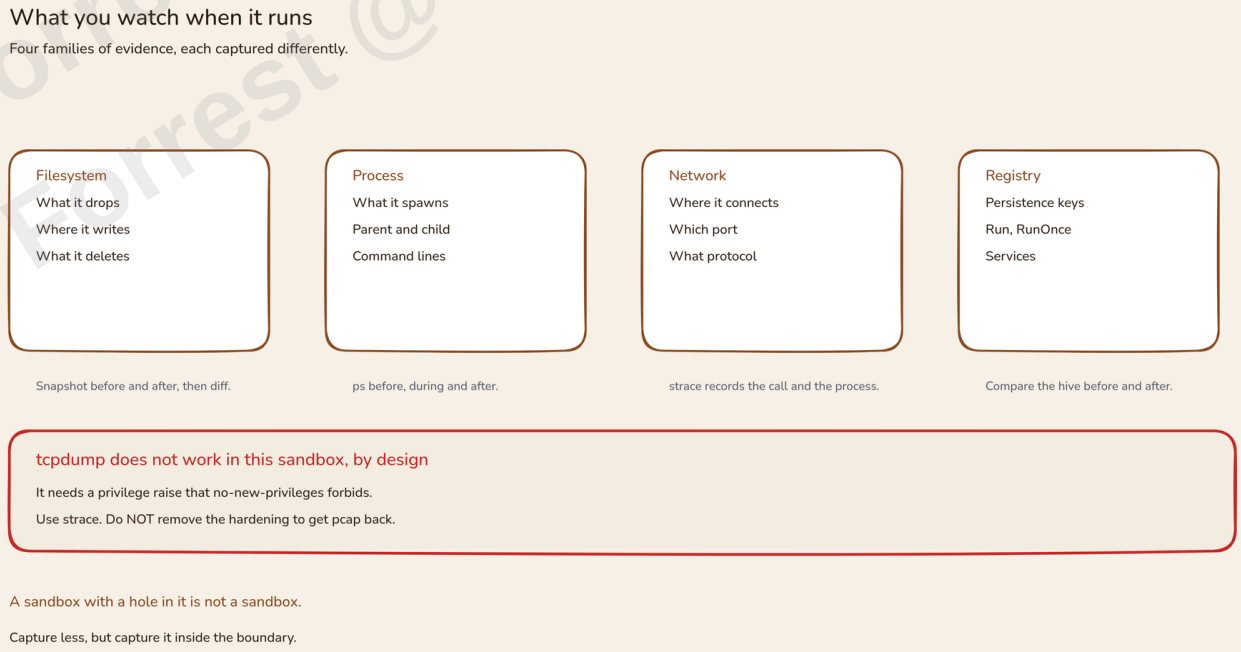


Figure 3 (repeated for reference): the four families of runtime evidence.

© Billy Forrest @ https://training.forrest.one

Family	Question	How to capture it
Filesystem	What did it create, change, delete?	Snapshot, run, diff
Process	What did it spawn?	ps before, during, after
Network	Where did it connect?	strace -e trace=connect
Registry	What persistence did it set?	Compare the hive before and after

The filesystem diff is the highest-value artefact of a dynamic run. Capture the file tree before detonation and after, then diff. Everything new is attributable to the sample.

```
# before
find / -xdev -type f -newermt '-1 minute' > /tmp/before.txt
# ... detonate ...
# after
find / -xdev -type f -newermt '-1 minute' > /tmp/after.txt
diff /tmp/before.txt /tmp/after.txt
```

Capture volatile evidence first. Process lists, network connections and open files exist only while the sample runs. Once it exits, they are gone.

Then ./sandbox.sh logs harvests the victim's evidence into evidence/<timestamp>/ so it is preserved outside the container.

6.4 Reading a trace for intent

A syscall trace is a sequence, and sequence is the intelligence. Individually innocent calls become a story in combination:

Sequence	What it is
openat(O_RDONLY) on a browser profile, then connect()	Credential theft followed by exfiltration
openat(O_WRONLY|O_CREAT) across many files, then unlink()	Ransomware: write encrypted copies, delete originals
execve of a shell, then connect()	A reverse shell
ptrace() then repeated mmap	Anti-debugging or in-memory unpacking
connect() to many hosts in turn	Scanning or fast-flux C2

The pattern is the finding, not the individual call. openat appears in almost every program ever written. openat on a credential store immediately before a network connection does not.

7. Documents, scripts and archives

Not every sample is an executable. Office documents, PDFs and archives are the most common initial-access vector.

7.1 Office documents

oletools dissects OLE and OOXML files — macros, embedded objects, and the OLE streams inside them.

```
oleid report.docm          # risk summary
olevba report.docm         # extract and analyse VBA macros
mraptor report.docm        # macro risk triage
oleobj report.docm         # extract embedded objects
```


© Billy Forrest @ https://training.forrest.one

oleid first — it is the fast pass. It flags macros, encryption and suspicious external relationships without dumping all the content.

Then olevba, and read the macro for these four things:

Look for	What it means
AutoOpen, Document_Open	**The macro runs automatically.** The single biggest red flag
Shell, WScript.Shell	It will execute a command
CreateObject("MSXML2.XMLHTTP")	It will download something
Long strings of digits and Chr() calls	An obfuscated payload, built at run time

mraptor scores macro risk in one line, which is useful when you are triaging fifty documents rather than one.

An encrypted document cannot be analysed without the password. oleid reports encryption as a risk, and that is a legitimate finding: encrypting a document is also how you stop a scanner reading it.

7.2 PDFs

```
pdf-parser --stats report.pdf        # object summary
pdf-parser --search /JavaScript report.pdf
pdf-parser --search /OpenAction report.pdf
```

Three PDF features carry almost all the risk:

Feature	Why
/JavaScript	Runs code on open
/OpenAction	Automatically triggers an action
/Launch	Executes an external file

A PDF that is a delivery vehicle is usually mostly padding. Look at the object count and the file size together: a large file with very few visible objects has something hidden in it.

7.3 Archives

Malware arrives zipped, often password-protected to defeat scanning.

```
7z l sample.zip               # list without extracting
7z x sample.zip -ooutdir      # extract
7z l -slt sample.zip          # technical listing: compression method, CRC
```

Check the compression ratio. A small archive that expands enormously is a bomb or a dropper packed inside a dropper. 7z l -slt shows the ratio per file.

A password-protected archive is doing two jobs: stopping your scanner, and tempting the user into running what is inside after they type the password from the email body.

8. Reporting

An analysis that is not written up did not happen. The report is the deliverable.

Billy Forrest @ https://training.forrest.one

© Billy Forrest @ https://training.forrest.one

8.1 Structure

```
# Malware Analysis Report — <sample name>

## Summary
Three sentences. What it is, what it does, how dangerous it is.

## Sample identification
Filename, size, SHA-256, MD5, SHA-1, ssdeep, file type, architecture.

## Capabilities
What it can do, with the evidence for each claim.

## Behaviour (dynamic)
What it did when run — files, processes, network — with the trace.

## Indicators of compromise
Host-based, network-based, and detection rules.

## Detection
The YARA rule, and where it would be deployed.

## Limitations
What you could not determine, and what would change that.
```

8.2 The indicators table

Type	Indicator	Confidence	Source
Network	cdn.static-resources.test/microsoft-ph-update	**Confirmed**	strings
Host	\.\pipe\MSSE-2679-server	**Confirmed**	strings
Network	192.0.2.10:8443	**Confirmed**	strace
Host	/tmp/.beacon_sim_drop	**Confirmed**	strace
Behaviour	Reads /etc/passwd at startup	**Confirmed**	strace

State the method for every indicator. An indicator without a source cannot be challenged, and an indicator that cannot be challenged cannot be trusted.

8.3 Report the negatives

"No capabilities found by capa with the Linux backend on this 632-byte ELF" is a finding. It tells the reader that a specific tool was applied and returned nothing, which is different from nobody having looked. Put null results in the report with the method that produced them.

8.4 Verify before you deliver

```
pdfinfo report.pdf | grep -E 'Pages|Title|Author'
pdftotext report.pdf - | grep -c '<key indicator>'
```

Count the indicators in the PDF and compare against the source table. A report that renders is not the same as a report that is complete.

9. Tool reference

Every tool used in this guide, with the invocation that works in this lab.

Tool	Phase	Command
file	Identify	file <sample>

© Billy Forrest @ https://training.forrest.one

Tool	Phase	Command
sha256sum	Identify	sha256sum <sample>
ssdeep	Identify	ssdeep -b <sample>
strings	Static	strings -n 6 <sample>
rabin2	Static	rabin2 -I | -i | -S | -zz <sample>
xxd	Static	xxd <sample> | head
binwalk	Static	binwalk -e <sample>
upx	Static	upx -d <packed>
capa	Static	capa -r /opt/capa-rules --os linux <sample>
floss	Static	floss <sample> (PE and shellcode only)
yara	Detection	yara -w <rules.yar> <dir>/
oleid	Documents	oleid <doc>
olevba	Documents	olevba <doc>
mraptor	Documents	mraptor <doc>
oleobj	Documents	oleobj <doc>
pdf-parser	Documents	pdf-parser --stats <pdf>
7z	Archives	7z l -slt <archive>
/opt/maldev-tools/strace	Dynamic	strace -f -e trace=connect,openat,execve -o t.txt <sample>
objdump	RE	objdump -d <sample>
readelf	RE	readelf -a <sample>
gdb	RE	gdb -q <sample>
radare2	RE	r2 -A <sample>

Tool versions in this lab

Checked live, and cross-referenced against each project's upstream release page.



Screenshot: the version of every analysis tool installed in the lab, with its upstream source.

Tool	Version	Note
capa	9.4.0	Current release; adds Ghidra/PyGhidra support and 26 new rules
floss	3.1.1	Current FLARE release
yara	4.5.8	**YARA-X** is the Rust successor — near-100% rule compatible, much faster

© Billy Forrest @ https://training.forrest.one

Tool	Version	Note
radare2	6.0.5	The free disassembler that covers IDA's ground for triage
strace	7.0	The sandbox's primary network-capture tool
binwalk	2.4.3	Entropy and embedded-file carving
upx	4.2.4	The one packer you can usually reverse immediately
binutils	2.47	objdump, readelf, strings
gdb	17.2	Static use only here — the sandbox denies ptrace
ssdeep	2.14.1	Fuzzy hashing
7-Zip	26.03	Archive inspection
capa-rules	1057 files	The rule set capa needs when installed via pipx
Python	3.14.6	Runs the sandbox tooling

Verify versions yourself rather than trusting a guide, including this one. Tool behaviour changes between releases, and several of the fixes in the troubleshooting table are release-specific.

```
# one-line version audit of everything this guide uses
docker exec -u 0 kali-desktop bash -lc \
  'export PATH=$PATH:/usr/local/bin; capa --version; yara --version; floss --version'
```

Installing a missing tool in the lab

```
# apt — needs root
apt-get install -y yara ssdeep strace gdb

# Python CLI tools — PEP-668 blocked, use pipx with an explicit bindir
export PIPX_HOME=/opt/pipx PIPX_BIN_DIR=/usr/local/bin
pipx install flare-capa
pipx install oletools

# capa needs an explicit rule set when installed as a Python package
git clone --depth 1 https://github.com/mandiant/capa-rules /opt/capa-rules
```

Tools that are not in this lab, and why

Tool	Status	Note
tcpdump	**Does not work**	Blocked by no-new-privileges by design. Use strace.
strace	**Not on PATH**	Call /opt/maldev-tools/strace by full path.
Ghidra	Not installed	Needs a JVM; radare2 covers the same ground for triage
gdb	Installed	But the sandbox has no ptrace for a live process — static only
Volatility	Not installed	For memory images; see the sandbox's memory-triage tooling
idapro	Not installed	Commercial; radare2 is the free equivalent

10. Troubleshooting

Real failures hit while writing this guide, with the fix for each.

Symptom	Cause	Fix
strace: command not found in the sandbox	not on PATH	/opt/maldev-tools/strace

© Billy Forrest @ https://training.forrest.one

Symptom	Cause	Fix
capa: default embedded rules not found!	pipx install ships no rules	add -r /opt/capa-rules
capa: Input file does not appear to target a supported OS	OS not specified	add --os linux or --os windows
floss: FLOSS currently supports ... PE	ELF not supported	use strings and rabin2 -zz
tcpdump produces nothing	capability raise blocked	use strace; do not weaken the hardening
strings returns nothing readable	strings are obfuscated	try floss on PE, or unpack first
High entropy, few imports	the file is packed	unpack before disassembling
capstone import fails in the sandbox	wrong Python	use /opt/maldev-venv/bin/python
Analysis finds nothing at all	you may be looking at a stub	check for a second stage

The last row is worth stating plainly. Modern malware is frequently a small downloader whose only job is to fetch the real payload. If your sample is tiny, has almost no code, and only imports networking functions, the interesting file is not in front of you. Say so in the report rather than concluding the sample is benign.

11. Appendix — reproducing this guide

Every picture here is a capture of a real command run in the lab. Nothing was reconstructed. The tooling is included so any figure can be regenerated.

```
# 1. Run the real command, keeping the transcript
docker exec -u 0 kali-desktop bash -lc \
  'export PATH=$PATH:/usr/local/bin; <command>' \
  > evidence/$(date -u +%Y%m%dT%H%M%SZ)_<slug>.txt 2>&1

# 2. Screenshot the terminal that produced it
tools/term_shot.py images/<slug>.png <seconds> -- '<command>'

# 3. Crop the desktop chrome away so the text fills the page
tools/crop_shots.py images/*.png
```

Five pitfalls the capture tool exists to handle, each of which cost real time:

- qterminal has no title flag; set the window title with a printf escape and find the window with xdotool search --class qterminal.
- Resize the window before the command runs. The shell wraps output to the width it had when it printed, so maximising afterwards leaves every line broken mid-word.
- qterminal rejects --size and then never launches at all.
- Use pkill -x qterminal, not -f: the -f form matches the docker exec command line and kills the caller.
- The lab container has no Docker socket, so it cannot reach into the sandbox. The strace figure displays a capture made inside maldev-sandbox and labels it as such.

Billy Forrest @ https://training.forrest.one

© Billy Forrest @ https://training.forrest.one

What is in this folder

Path	Holds
samples/synthetic-triage-set/	The inert indicator-carrying samples
samples/known-good/	Structurally valid benign baselines
samples/beacon_sim and beacon_sim.c	The benign dynamic-analysis specimen and its source
samples/yara_rules.yar	The detection rules used in section 5.7
figures/	Excalidraw sources and rendered diagrams
images/	The terminal captures
evidence/	Raw transcripts and the sandbox strace output
tools/	The capture, crop, figure-build and PDF-render tooling

Training at Forrest.one · Malware Inspection & Reverse Engineering · Lab use, inert samples only.

Billy Forrest @ https://training.forrest.one

© Billy Forrest @ https://training.forrest.one